COMPSCI 308 Design and Analysis of Algorithms

Quiz 03

Xing Shi Cai

2026-02-13

👀 Don't Cheat

During the quiz/exam do *not*

- 🩹 give or receive aid;
- ⏱️ read the questions before the quiz begins;
- 🛑 keep writing when the quiz ends;
- 📱 use a phone, calculator, computer or the internet;
- 🚽 hide textbooks or other aids in the toilet.

⚡ Follow Rules

- 👨‍🎓 Enter your name and **SID** (not your Net ID) in the designated space below.
- 🆎 Support your proofs with both equations and descriptive reasoning.
- ✏️ Use only pen for writing as pencil marks are not clearly visible.
- ❓ Restrict your inquiries to clarifications regarding typos.

Name: SID:

Figure 1: Good Luck

1 Unbounded Feasible Region (1pt)

Give an example of a linear program in two variables x_1 and x_2 for which the feasible region is not bounded, but the optimal objective value is finite.

Answer:

One example is the following linear program:

$$\begin{aligned} \text{minimize} \quad & x_1 \\ \text{subject to} \quad & x_1 \geq 1 \\ & x_2 \geq 0 \end{aligned}$$

The feasible region is unbounded because x_1 can be arbitrarily large and x_2 can be arbitrarily large. However, the optimal objective value is 1, which is finite (achieved when $x_1 = 1$, for example at $(x_1, x_2) = (1, 0)$).

Rubrics

- Any valid linear program with an unbounded feasible region and a finite optimal objective value receives full credit.
- A common mistake is to give a program whose feasible region is bounded or whose optimal value is unbounded (i.e., $-\infty$ or $+\infty$).

2 Single-Source Shortest Paths as a Linear Program (1pt)

Given the following weighted directed graph G with source vertex s, write a linear program whose solution gives the shortest-path weights d_v from s to every vertex v.

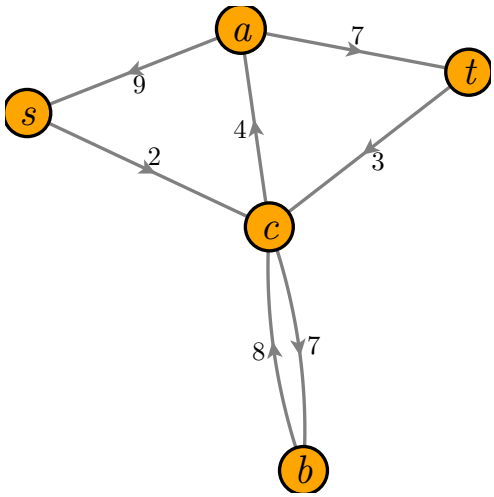

Figure 2: A weighted directed graph with source s.

Write down the LP with an explicit objective function and one constraint per edge, and include the source constraint $d_s = 0$. Use variable names d_s, d_t, d_a, d_b, d_c.

Answer:

The linear program should be

$$\begin{aligned}
\text{maximize} \quad & d_s + d_t + d_a + d_b + d_c \\
\text{subject to} \quad & d_c \leq d_s + 2 \\
& d_c \leq d_t + 3 \\
& d_s \leq d_a + 9 \\
& d_t \leq d_a + 7 \\
& d_c \leq d_b + 8 \\
& d_a \leq d_c + 4 \\
& d_b \leq d_c + 7 \\
& d_s = 0 \\
& d_t, d_a, d_b, d_c \geq 0
\end{aligned}$$

Key ideas.

- We **maximize** $\sum_v d_v$ so that each d_v is pushed as large as possible while still respecting the triangle-inequality constraints.
- The constraint $d_s = 0$ fixes the source distance.
- The non-negativity constraints $d_v \geq 0$ are required because shortest-path weights are non-negative.
- Every feasible d_v is a lower bound on the true shortest-path weight $\delta(s, v)$; maximizing forces $d_v = \delta(s, v)$.

3 Maximum Flow as a Linear Program (1pt)

Write out explicitly the linear program corresponding to finding the maximum flow in the following network.

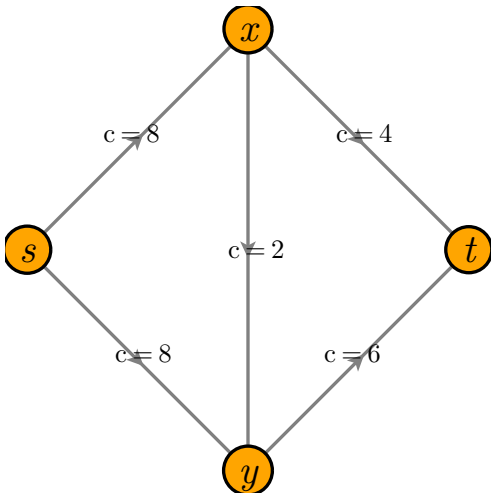

Figure 3: A flow network with edge capacities.

Use a flow variable f_{uv} for each edge (u, v).

Answer:

The linear program is

$$\begin{aligned}
\text{maximize} \quad & f_{sx} + f_{sy} \\
\text{subject to} \quad & f_{sx} = f_{xy} + f_{xt} \\
& f_{sy} + f_{xy} = f_{yt} && \text{(flow conservation)} \\
& f_{sx} \leq 8 \\
& f_{sy} \leq 8 \\
& f_{xy} \leq 2 \\
& f_{xt} \leq 4 \\
& f_{yt} \leq 6 && \text{(capacity constraints)} \\
& f_{sx},\ f_{sy},\ f_{xy},\ f_{xt},\ f_{yt} \geq 0
\end{aligned}$$

Key ideas.

- The objective maximizes the total flow leaving the source s.
- Flow conservation requires that, for every internal vertex $v \notin \{s, t\}$, the total flow into v equals the total flow out of v.
- Each flow variable is bounded above by the corresponding edge capacity and bounded below by 0.

4 Formulating the Dual (1pt)

Consider the following linear program in standard form:

$$
\begin{aligned}
\text{maximize} \quad & 8x_1 + 8x_2 + 2x_3 \\
\text{subject to} \quad & 4x_1 + 3x_2 + 4x_3 \leq 13, \\
& 6x_1 + 5x_2 + 3x_3 \leq 26, \\
& 5x_1 + 2x_2 + 1x_3 \leq 17 \\
& x_1,\ x_2,\ x_3 \geq 0.
\end{aligned}
$$

Formulate its dual.

Answer:

The dual is

$$
\begin{aligned}
\text{minimize} \quad & 13y_1 + 26y_2 + 17y_3 \\
\text{subject to} \quad & 4y_1 + 6y_2 + 5y_3 \geq 8, \\
& 3y_1 + 5y_2 + 2y_3 \geq 8, \\
& 4y_1 + 3y_2 + 1y_3 \geq 2 \\
& y_1,\ y_2,\ y_3 \geq 0.
\end{aligned}
$$

How to construct the dual.

1. The primal has 3 constraints and 3 variables, so the dual has 3 constraints and 3 variables $y_1, \dots, y_3$.
2. The primal maximizes; the dual minimizes.
3. The dual objective uses the primal RHS values b_i as coefficients.
4. Each dual constraint j uses column j of the constraint matrix A with direction $\geq$ and RHS equal to c_j.
5. All dual variables are ≥ 0.

5 Randomized Algorithms (1pt)

Rank

The **rank** of a candidate is their quality relative to all other candidates. A candidate with rank k is the k-th best out of all n candidates. Rank n is the best; rank 1 is the worst.

Consider the following hiring algorithm:

```
Hire-Detective()
best = 0                                   ▷ A dummy candidate
for i ← 1 to n do
    interview candidate i
    if candidate i is better than the current best candidate then
        best = i
        hire candidate i to be a 🕵
```

Candidate i	1	2	3	4	5	6	7	8	9	10
Rank	2	9	7	1	4	8	6	10	5	3

How many candidates will be hired to become a 🕵?

Answer:

$$\boxed{3}$$

We trace through the ranks in interview order and hire whenever we see a new maximum:

Candidate 1: rank 2 🕵️ **hired**\Candidate 2: rank 9 🕵️ **hired**\Candidate 3: rank 7\Candidate 4: rank 1\Candidate 5: rank 4\Candidate 6: rank 8\Candidate 7: rank 6\Candidate 8: rank 10 🕵️ **hired**\Candidate 9: rank 5\Candidate 10: rank 3

Total hired: 3.

6 Quicksort (1 pt)

👀 QUICKSORT and PARTITION Algorithms

```
QUICKSORT(A, p, r)
if p < r then
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q − 1)
    QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
x = A[r]
i = p − 1
j = p
while j ≤ r − 1 do
    if A[j] ≤ x then
        i = i + 1
        swap A[i] with A[j]
    j = j + 1
swap A[i + 1] with A[r]
return i + 1
```

Trace the execution of PARTITION(A, p, r) on the array below. Show the array after **each iteration** of the while loop and mark the positions of i and j. After the loop, show the **final** array and mark the pivot index q.

💡 The initial state and the state after one iteration are given as examples.

Initial state (pivot $x = 3$)

i	j, p						r
2	5	7	1	4	8	6	3

After iteration 1

i, p	j						r
2	5	7	1	4	8	6	3

After iteration 2

i, p		j					r
2	5	7	1	4	8	6	3

After iteration 3

i, p			j				r
2	5	7	1	4	8	6	3

After iteration 4

p	i			j			r
2	1	7	5	4	8	6	3

After iteration 5

p	i				j		r
2	1	7	5	4	8	6	3

After iteration 6

p	i					j	r
2	1	7	5	4	8	6	3

After iteration 7

p	i						j, r
2	1	7	5	4	8	6	3

Final array

p		q					r
2	1	3	5	4	8	6	7

7 Hash Tables (1pt)

Independent Uniform Hashing

An **ideal** hashing function h maps each key $k \in U$ to a slot in $\{0, 1, \dots, m-1\}$ so that:

- $h(k)$ is chosen **uniformly** from the m slots,
- choices for different keys are **independent**, and
- once chosen, the value of $h(k)$ stays fixed for that key.

Such a function is called an **independent uniform hash function** (also called a random oracle). Using a hash table under this model is called **independent uniform hashing**.

You use a hash function h to hash n distinct keys into an array T of length m. Assuming independent uniform hashing, let

$$X = |\{\{k_1, k_2\} : k_1 \neq k_2 \text{ and } h(k_1) = h(k_2)\}| .$$

7.1 (0.5 pt)

What is $\mathbb{E}[X]$?

Answer:

$$\boxed{\frac{n(n-1)}{2m}}$$

7.2 (0.5 pt)

Prove your answer.

Answer:

For each unordered pair of distinct keys $\{k_i, k_j\}$, define

$$X_{ij} = \begin{cases} 1, & h(k_i) = h(k_j), \\ 0, & \text{otherwise.} \end{cases}$$

Then the number of collisions is

$$X = \sum_{1 \le i < j \le n} X_{ij}.$$

Under independent uniform hashing,

$$\mathbb{P}(h(k_i) = h(k_j)) = \frac{1}{m},$$

so

$$\mathbb{E}[X_{ij}] = \frac{1}{m}.$$

By linearity of expectation,

$$\mathbb{E}[X] = \sum_{1 \le i < j \le n} \mathbb{E}[X_{ij}] = \binom{n}{2} \frac{1}{m} = \frac{n(n-1)}{2m}.$$

Therefore, the expected number of collisions is

$$\boxed{\frac{n(n-1)}{2m}}.$$

8 Hash Tables (1pt)

💡 Adapted from Exercise 11.4-1 in *Introduction to Algorithms*.

Consider inserting the keys 65, 37, 7, 1, 96, 94, 75, 83, 64 in the given order into a hash table of length $m = 11$ using open addressing with linear probing $h(k, i) = (k + i) \bmod m$. Give only the final table after all keys are inserted.

Answer:

A correct final table (slots 0 through 10) is:

0	1	2	3	4	5	6	7	8	9	10
83	1	64		37		94	7	96	75	65

The End of The Quiz